



Project: Computing the first input of the $3k + 1$ sequence that has $\geq$ 9 terms

Course: COEN 313

Name: Bryce Orchard

Student ID: 40336764

Due Date: June 29th, 2026

Current Date: June 28th, 2026

"I certify that this submission is my original work and meets the Faculty's Expectations of
Originality"

A handwritten signature in black ink, appearing to be "Bryce Orchard".

Table of Contents

Introduction.....	3
Part 1.....	4
Code	4
Simulation	6
Synthesis	8
Part 2.....	9
ASM Chart (Moore).....	9
ASM Block Diagram	10
Code	10
Simulation	13
Synthesis	13
References	14
Appendix	15
Appendix A	15
Appendix B	16

Introduction

The Collatz Conjecture [1] is a famous unsolved math problem:

$$\text{Given } f(n) = \begin{cases} n/2, & n \text{ even} \\ 3n + 1, & n \text{ odd} \end{cases}$$

Form a sequence by performing this operation repeatedly, and taking the result at each step as the input at the next:

$$a_i = \begin{cases} n & \text{for } i = 0 \\ f(a_{i-1}) & \text{for } i > 0 \end{cases}$$

The conjecture states that for any $n \in \mathbb{N}$, there exists an i such that $a_i = 1$.

This project considers a much simpler case, which is to find the smallest n that has ≥ 9 terms in its ‘3k plus one’ sequence. An example for $n = 3$:

$$n = 3: \{3, 10, 5, 16, 8, 4, 2, 1\}$$

Two different solutions were written in VHDL language and synthesized to the NEXYS A7 FPGA board [2]. The first employs a single clocked process that calculates the terms of the sequence using variables and a simple algorithm. The second solution uses a Moore-style FSM to calculate the next state based on inputs, and calculate the outputs to the data path based on the current state. Both solutions were successfully implemented and synthesized in Vivado to the A7 board, and their core functionality was verified in ModelSim.

The following libraries and entity were used for both solutions:

```
library IEEE;
use ieee.numeric_std.all;
use IEEE.std_logic_1164.all;

entity three_k_plus_one is
    port(reset : in std_logic; -- asynchronous
          clk_in : in std_logic; -- the 100MHz FPGA board clock
          an: out std_logic_vector(7 downto 0); -- the 8 active low anodes of each 7-seg
          -- display
          sseg : out std_logic_vector(7 downto 0); -- the 8 active low cathodes of each 7-seg
          -- display
          done_out : out std_logic);
end three_k_plus_one;
```

The XDC file used for both solutions can be found at Appendix A.

Part 1

Code

The following C algorithm was used as inspiration for the VHDL implementation:

```
// Author: Ted Obuchowicz
// Oct. 10, 2002
// file: 3k+1.C

#include <iostream>
using namespace std;

int main() {
    unsigned long int number = 1;
    unsigned long int term = 1;
    unsigned long int length = 1;
    do {
        number = number + 1;
        term = number;
        length = 1;
        cout << "Testing number = " << number << endl;

        while ( term != 1) {
            if (term % 2 == 0 ) term = term / 2 ;
            else term = (3*term + 1);
            cout << " " << term ;
            length = length + 1;
        };
        cout << endl << endl;

    } while (length < 100);

    cout << number << endl;
    return 0;
}
```

The VHDL code for the algorithmic solution below is split up into two sections: the main logic, and the 7-segment driver:

```
architecture structural of three_k_plus_one is
    signal done : std_logic; -- create signal for done since it needs to be read
    signal number_out : unsigned(3 downto 0);
    signal term_out : unsigned(6 downto 0);

    -- ----- 7-segment -----
    -- The 7-segment uses a counter to create the necessary delay for multiplexing the 3 hex
    -- values to be displayed (number, term_out0, term_out1)
    -- The size of the counter is given by:
    --  $N = \log_2(\text{clock frequency} / \text{target LED refresh rate}) = \log_2(100 \text{ MHz} / 90 \text{ Hz}) = 19.9 \sim 20$ 
    -- 90 Hz = 10.5 ms cycle time for each of the 4 hex values to be displayed, which is fast
    -- enough to appear continuous to the human eye
    constant N : integer := 20;
    signal q_reg, q_next : unsigned(N-1 downto 0);
    signal sel : std_logic_vector(1 downto 0);
    signal hex : std_logic_vector(3 downto 0);
```

```

begin
  LOGIC : process(clk_in, reset)
    -- use 4 bits for the possible integers (we expect a max of 6)
    variable number, length : unsigned(3 downto 0) := "0001";
    -- use 7 bits for the terms of the sequence
    variable term : unsigned(6 downto 0) := "0000001";
  begin
    if reset = '1' then
      number := "0001";
      length := "0001";
      term := "0000001";
      done <= '0';
    elsif rising_edge(clk_in) then
      if done /= '1' then
        if length = "1001" then -- if we have reached 9 terms, we are done
          done <= '1';
          -- if term = 1, we have completed the sequence for the current number and can
          -- move to the next number
          elsif term = "0000001" then
            number := number + 1;
            term := resize(number, 7);
            length := "0001"; -- reset length to 1 since we have already counted the
                               -- first term of the new sequence
          else
            if term(0) = '0' then -- if even
              term := term / 2;
            else -- if odd
              term := resize(3*term, 7) + 1;
            end if;
            length := length + 1;
          end if;
        end if;
      end if;
      -- update the outputs to be displayed on the 7-segment LEDs
      number_out <= number;
      term_out <= term;
    end process LOGIC;

    done_out <= done;

    -- register for the counter to create the necessary delay for the 7-segment display
    -- multiplexing
    COUNTER_REG : process(clk_in, reset)
    begin
      if reset = '1' then
        q_reg <= (others => '0');
      elsif rising_edge(clk_in) then
        q_reg <= q_next;
      end if;
    end process COUNTER_REG;

    -- next-state logic for the counter
    q_next <= q_reg + 1;

    -- 2 MSBs of counter to control 4-to-1 multiplexing
    sel <= std_logic_vector(q_reg(N-1 downto N-2));

    -- asynchronous multiplexing of the 4 hex values to be displayed
    HEX_MUX : process(sel, term_out, number_out)
    begin
      case sel is
        when "00" =>
          an <= "11111110";
          hex <= std_logic_vector(number_out);
        when "01" =>
          an <= "11101111";
          -- display the 4 least significant bits of term as the first hex value

```

```

        hex <= std_logic_vector(term_out(3 downto 0));
when others => -- "10" and "11" both display the second hex value since we only
               -- have 3 hex values
        an <= "11011111";
        -- display the 3 most significant bits of term as the second hex value (with a
        -- leading 0)
        hex <= std_logic_vector('0' & term_out(6 downto 4));
    end case;
end process HEX_MUX;

-- hex-to-7-segment LED decoding
with hex select
    sseg <= -- sseg format is [a, b, c, d, e, f, g, dp]
        "00000011" when "0000", -- 0
        "10011111" when "0001", -- 1
        "00100101" when "0010", -- 2
        "00001101" when "0011", -- 3
        "10011001" when "0100", -- 4
        "01001001" when "0101", -- 5
        "01000001" when "0110", -- 6
        "00011111" when "0111", -- 7
        "00000001" when "1000", -- 8
        "00001001" when "1001", -- 9
        "00010001" when "1010", -- a
        "11000001" when "1011", -- b
        "01100011" when "1100", -- c
        "10000101" when "1101", -- d
        "01100001" when "1110", -- e
        "01110001" when others; -- f

end structural;

```

The algorithm for this solution is quite simple. First check if we're done. If not, check if the length (number of terms in the sequence) is 9. If it is, we're done. If not, check if the term is equal to one. If it is, reset the length to 1 and continue with the next number. If not, increment the length and perform the function $f(n)$ on the term.

Variable were used for number, term, and length so that the incremented value of number could immediately be loaded into term within the same clock cycle, mirroring the C++ behaviour. If number were a signal, it would require an additional clock cycle for the incremented value to propagate (as seen below in the Moore FSM solution). While number needs to be a variable, length and term could be represented as a signal and the algorithm would still hold. However, if length and term were signals and assigned within the rising_edge block, during simulation, their values would lag by one clock cycle.

The 7-segment multiplexing functionality has been modified from Pong P. Chu's work [3, pp. 88-95] in order to work on the A7 board. Notably, the constant N was changed to 20 to achieve a 90 Hz refresh rate. The case statement in the HEX_MUX process was modified for the purposes of this algorithm. Also, the least significant bit of sseg was set to the constant '1' to force the decimal point off.

Simulation

The ModelSim simulation run used a modified version shown below, which removes all 7-segment logic, and adds a length signal so that it could be shown in the simulation:

```

architecture structural of three_k_plus_one is
    signal done : std_logic;

```

```

signal number_out, length_out : unsigned(3 downto 0);
signal term_out : unsigned(6 downto 0);

begin
  process(clk_in, reset)
    variable number, length : unsigned(3 downto 0) := "0001";
    variable term : unsigned(6 downto 0) := "0000001";
  begin
    if reset = '1' then
      number := "0001";
      length := "0001";
      term := "0000001";
      done <= '0';
    elsif rising_edge(clk_in) then
      if done /= '1' then
        if length = "1001" then
          done <= '1';
        elsif term = "0000001" then
          number := number + 1;
          term := resize(number, 7);
          length := "0001";
        else
          if term(0) = '0' then
            term := term / 2;
          else
            term := resize(3*term, 7) + 1;
          end if;
          length := length + 1;
        end if;
      end if;
    end if;
    number_out <= number;
    term_out <= term;
    length_out <= length;
  end process;

  done_out <= done;
end structural;

```

The .do file used for simulation:

```

add wave clk_in
add wave reset
add wave number_out
add wave term_out
add wave length_out
add wave done_out

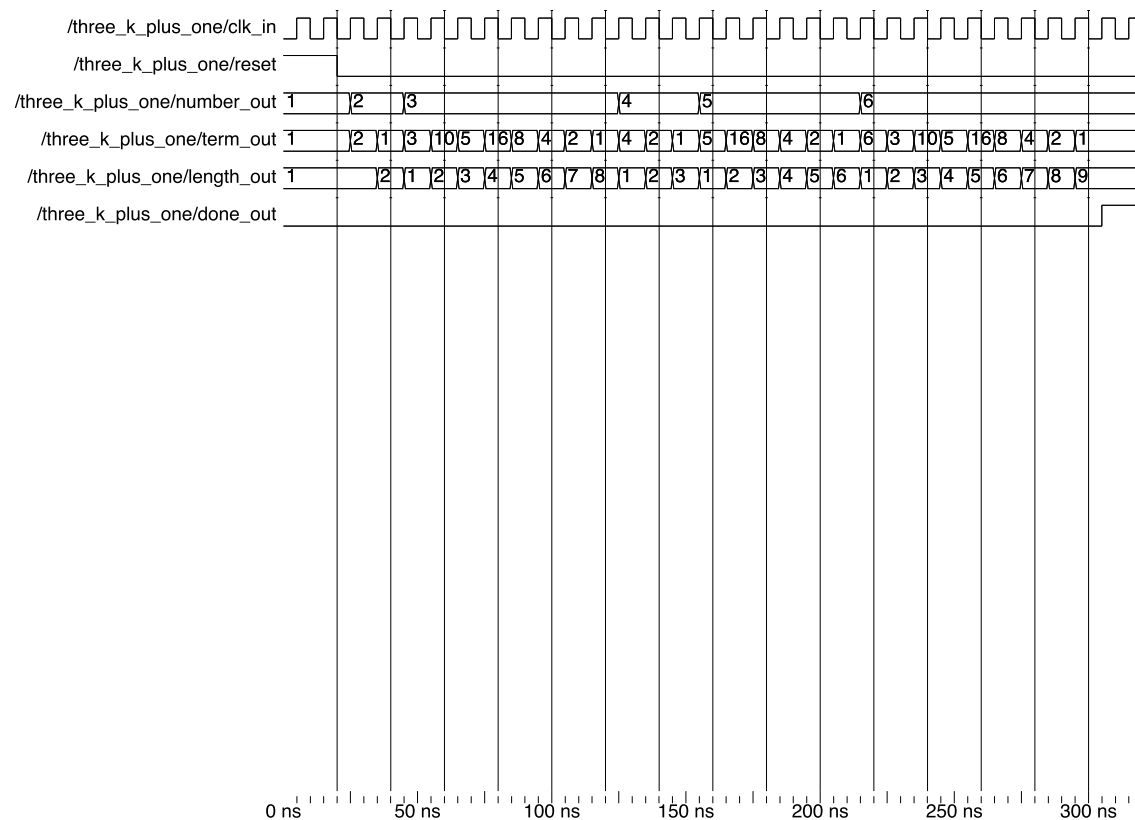
# Generate clock: period 10ns, 50% duty cycle
force clk_in 0 0, 1 5 -repeat 10

# Assert reset for 2 clock cycles
force reset 1
run 20

# Start
force reset 0
run 320

```

The ModelSim results:



Entity: three_k_plus_one Architecture: structural Date: Fri Jun 26 06:37:24 PM EDT 2026 Row: 1 Page: 1

Of note: Since the length signal is never read by the program, ModelSim automatically optimizes it away, therefore, the vsim command must be run with the -voptargs="+acc" option to prevent such optimizations.

Synthesis

The complete solution (without the length signal and includes 7-segment logic) was synthesized to the A7 board, and a .bit file was generated. The timestamp information for the .bit file is shown below:

```
Terminal
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > pwd
/nfs/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 b_orchar b_orchar 3825917 Jun 26 16:57 three_k_plus_one.bit
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > whoami
b_orchar
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] >
```

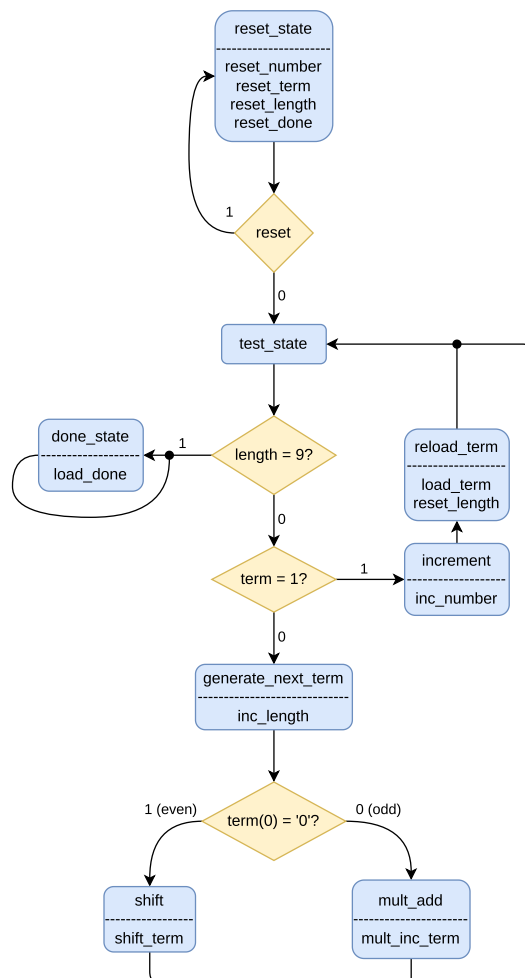

During implementation, 0 Warnings, 0 Critical Warnings, and 0 Errors were generated. Synthesis gave the following warnings:

WARNING: [Synth 8-7080] Parallel synthesis criteria is not met
 WARNING: [Synth 8-3917] design three_k_plus_one has port an[7] driven by constant 1
 WARNING: [Synth 8-3917] design three_k_plus_one has port an[6] driven by constant 1
 WARNING: [Synth 8-3917] design three_k_plus_one has port an[3] driven by constant 1
 WARNING: [Synth 8-3917] design three_k_plus_one has port an[2] driven by constant 1
 WARNING: [Synth 8-3917] design three_k_plus_one has port an[1] driven by constant 1

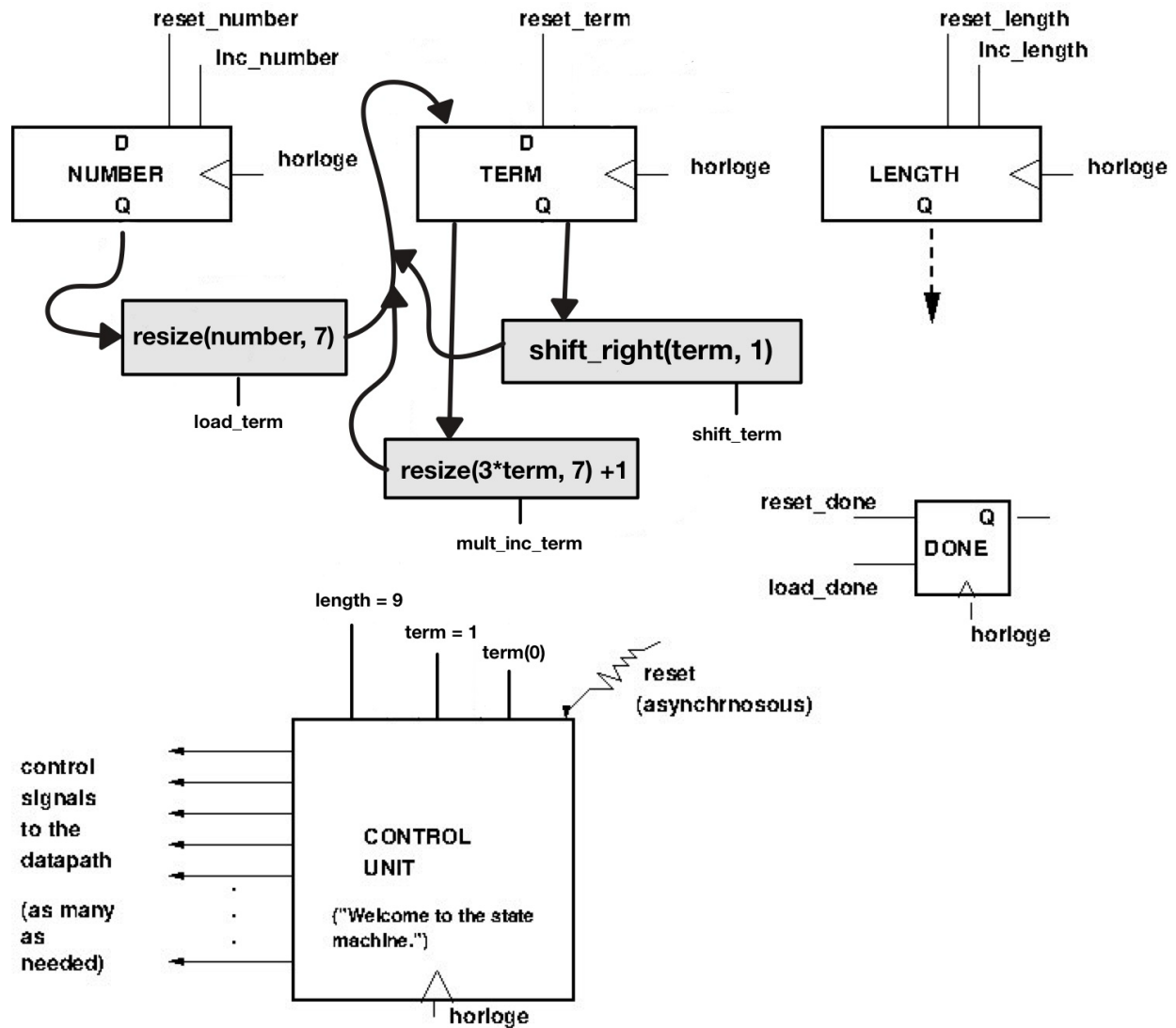
Synth 8-7080 occurs only because the design is too small to meet the parallel synthesis criteria, and has no impact on the correctness of synthesis. All Synth 8-3917 warnings are due to 5 of the 7-segment displays being unused and therefore tied to 1. This is by design, so the warnings can be disregarded.

Part 2

ASM Chart (Moore)



ASM Block Diagram



Code

The VHDL code for the Moore FSM solution:

```
architecture structural of three_k_plus_one is
-- data path registers
signal done : std_logic;
signal number, length : unsigned(3 downto 0);
signal term : unsigned(6 downto 0);

-- status signals
signal reset_number, inc_number : std_logic; -- number input signals

signal reset_term, shift_term, mult_inc_term, load_term : std_logic; -- term input signals

signal reset_length, inc_length : std_logic; -- length input signals

signal reset_done, load_done : std_logic; -- done input signals
```

```

type STATE_TYPE is (reset_state, test_state, increment, reload_term, generate_next_term,
shift, mult_add, done_state);
signal state : STATE_TYPE;

-- ----- 7-segment -----
-- The 7-segment uses a counter to create the necessary delay for multiplexing the 3 hex
-- values to be displayed (number, term_out0, term_out1)
-- The size of the counter is given by:
--  $N = \log_2(\text{clock frequency} / \text{target LED refresh rate}) = \log_2(100 \text{ MHz} / 90 \text{ Hz}) = 19.9 \sim 20$ 
-- 90 Hz = 10.5 ms cycle time for each of the 4 hex values to be displayed, which is fast
-- enough to appear continuous to the human eye
constant N : integer := 20;
signal q_reg, q_next : unsigned(N-1 downto 0);
signal sel : std_logic_vector(1 downto 0);
signal hex : std_logic_vector(3 downto 0); -- value to be displayed at any given moment

begin
  LOGIC : process(clk_in, reset) -- control logic
  begin
    if reset = '1' then
      state <= reset_state;
    elsif rising_edge(clk_in) then
      case state is
        when reset_state =>
          state <= test_state; -- reset must be '0'
        when test_state =>
          if length = "1001" then
            state <= done_state;
          elsif term = "0000001" then
            state <= increment;
          else
            state <= generate_next_term;
          end if;
        when increment =>
          state <= reload_term;
        when reload_term =>
          state <= test_state;
        when generate_next_term =>
          if term(0) = '0' then
            state <= shift;
          else
            state <= mult_add;
          end if;
        when shift =>
          state <= test_state;
        when mult_add =>
          state <= test_state;
        when done_state =>
          state <= done_state;
      end case;
    end if;
  end process LOGIC;

  OUTPUTS : process(state) -- non registered outputs
  begin
    reset_number <= '0'; inc_number <= '0';
    reset_term <= '0'; shift_term <= '0';
    mult_inc_term <= '0'; load_term <= '0';
    reset_length <= '0'; inc_length <= '0';
    reset_done <= '0'; load_done <= '0';
    case state is
      when reset_state =>
        reset_number <= '1'; reset_term <= '1';
        reset_length <= '1'; reset_done <= '1';
      when test_state =>
        null;
      when increment =>
        inc_number <= '1';
    end case;
  end process OUTPUTS;
end;

```

```

        when reload_term =>
            load_term    <= '1';
            reset_length <= '1';
        when generate_next_term =>
            inc_length    <= '1';
        when shift =>
            shift_term    <= '1';
        when mult_add =>
            mult_inc_term <= '1';
        when done_state =>
            load_done     <= '1';
    end case;
end process OUTPUTS;

NUMBER_PROCESS : process(clk_in) -- number
begin
    if rising_edge(clk_in) then
        if reset_number = '1' then
            number <= "0001";
        elsif inc_number = '1' then
            number <= number + 1;
        end if;
    end if;
end process NUMBER_PROCESS;

TERM_PROCESS : process(clk_in) -- term
begin
    if rising_edge(clk_in) then
        if reset_term = '1' then
            term <= "0000001";
        elsif shift_term = '1' then
            term <= shift_right(term, 1);
        elsif mult_inc_term = '1' then
            term <= resize(3*term, 7) + 1;
        elsif load_term = '1' then
            term <= resize(number, 7);
        end if;
    end if;
end process TERM_PROCESS;

LENGTH_PROCESS : process(clk_in) -- length
begin
    if rising_edge(clk_in) then
        if reset_length = '1' then
            length <= "0001";
        elsif inc_length = '1' then
            length <= length + 1;
        end if;
    end if;
end process LENGTH_PROCESS;

DONE_PROCESS : process(clk_in) -- done
begin
    if rising_edge(clk_in) then
        if reset_done = '1' then
            done <= '0';
        elsif load_done = '1' then
            done <= '1';
        end if;
        -- can be expressed as done <= (load_done or done) and not reset_done; but the
        -- above is more concise and easier to read
    end if;
end process DONE_PROCESS;
done_out <= done;

-- register for the counter to create the necessary delay for the 7-segment display
-- multiplexing

```

```

COUNTER_REG : process(clk_in, reset)
begin
    if reset = '1' then
        q_reg <= (others => '0');
    elsif rising_edge(clk_in) then
        q_reg <= q_next;
    end if;
end process COUNTER_REG;

q_next <= q_reg + 1;
sel <= std_logic_vector(q_reg(N-1 downto N-2));

-- asynchronous multiplexing of the 4 hex values to be displayed
HEX_MUX : process(sel, number, term)
begin
    case sel is
        when "00" =>
            an <= "11111110";
            hex <= std_logic_vector(number);
        when "01" =>
            an <= "11101111";
            hex <= std_logic_vector(term(3 downto 0));
        when others => -- "10" and "11" both display the second hex value since we only
                        -- have 3 hex values
            an <= "11011111";
            hex <= '0' & std_logic_vector(term(6 downto 4));
    end case;
end process HEX_MUX;

-- hex-to-7-segment LED decoding
with hex select
    sseg <=
        "00000011" when "0000", -- 0
        "10011111" when "0001", -- 1
        "00100101" when "0010", -- 2
        "00001101" when "0011", -- 3
        "10011001" when "0100", -- 4
        "01001001" when "0101", -- 5
        "01000001" when "0110", -- 6
        "00011111" when "0111", -- 7
        "00000001" when "1000", -- 8
        "00001001" when "1001", -- 9
        "00010001" when "1010", -- a
        "11000001" when "1011", -- b
        "01100011" when "1100", -- c
        "10000101" when "1101", -- d
        "01100001" when "1110", -- e
        "01110001" when others; -- f
end structural;

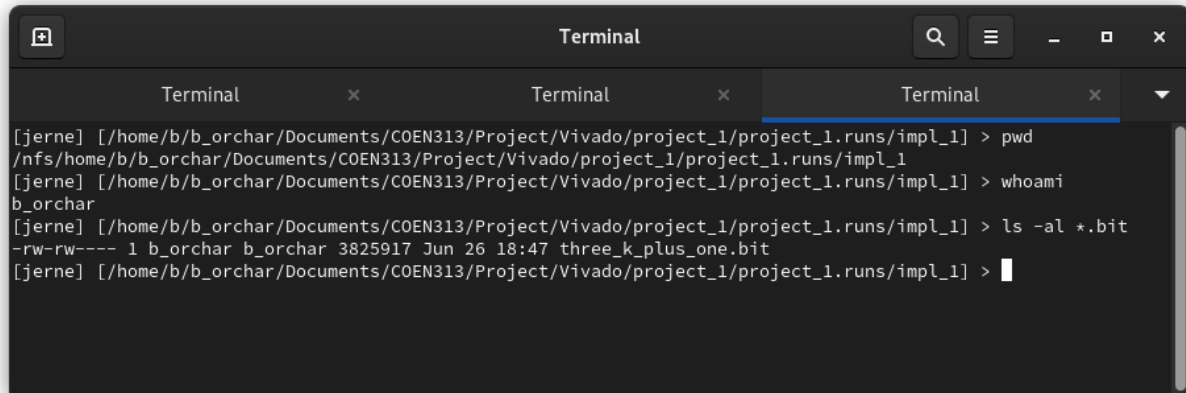
```

Simulation

The ModelSim simulation results were split into 150 ns sections for better readability. They can be found at Appendix B.

Synthesis

The timestamp information for the .bit file is shown below:

A screenshot of a terminal window with a dark background. The window title is "Terminal". It shows a series of commands and their outputs. The first command is 'pwd', which outputs the full path to the current directory. The second command is 'whoami', which outputs the username 'b_orchar'. The third command is 'ls -al *.bit', which lists a file named 'three_k_plus_one.bit' with its permissions, owner, group, size, date, and filename. The prompt is '[jerne]' and the directory is '/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1'.

```
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > pwd
/nfs/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > whoami
b_orchar
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > ls -al *.bit
-rw-rw---- 1 b_orchar b_orchar 3825917 Jun 26 18:47 three_k_plus_one.bit
[jerne] [/home/b/b_orchar/Documents/COEN313/Project/Vivado/project_1/project_1.runs/impl_1] > 
```

During implementation, 0 Warnings, 0 Critical Warnings, and 0 Errors were generated. Synthesis gave the following warnings:

```
WARNING: [Synth 8-7080] Parallel synthesis criteria is not met
WARNING: [Synth 8-3917] design three_k_plus_one has port an[7] driven by constant 1
WARNING: [Synth 8-3917] design three_k_plus_one has port an[6] driven by constant 1
WARNING: [Synth 8-3917] design three_k_plus_one has port an[3] driven by constant 1
WARNING: [Synth 8-3917] design three_k_plus_one has port an[2] driven by constant 1
WARNING: [Synth 8-3917] design three_k_plus_one has port an[1] driven by constant 1
```

References

- [1] Wikipedia, "Collatz Conjecture," [Online]. Available: https://en.wikipedia.org/wiki/Collatz_conjecture. [Accessed 28 June 2026].
- [2] Digilent, "Nexys A7 Reference Manual," [Online]. Available: <https://digilent.com/reference/programmable-logic/nexys-a7/reference-manual>. [Accessed 28 June 2026].
- [3] P. P. Chu, FPGA Prototyping by VHDL Examples: Xilinx Spartan-3 Version, Ohio: WILEY, 2008, p. 468.

Appendix

Appendix A

```
# XDC Constraints for three_k_plus_one on Nexys A7
# clk_in    -> E3 (100 MHz clock)
# reset     -> BTNC
# done_out  -> LD15
# sseg[7:0] -> CA:DP
# an[7:0]   -> AN7:AN0

## Clock signal -> E3
set_property -dict { PACKAGE_PIN E3      IOSTANDARD LVCMOS33 } [get_ports { clk_in }];
## Create clock to suppress Vivado warnings
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports { clk_in }];

## reset -> BTNC
set_property -dict { PACKAGE_PIN N17     IOSTANDARD LVCMOS33 } [get_ports { reset }];

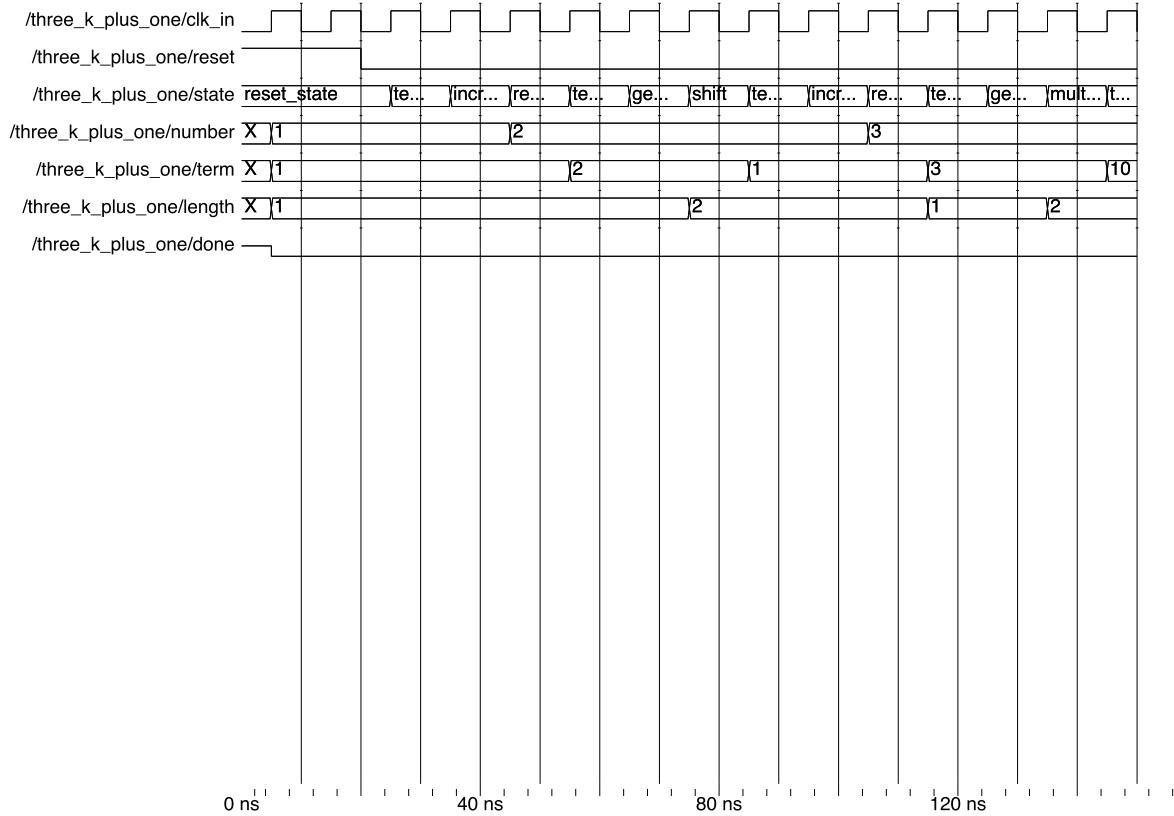
## done_out -> LD15
set_property -dict { PACKAGE_PIN V11     IOSTANDARD LVCMOS33 } [get_ports { done_out }];

## 7 segment display segments | seg[7:0] -> CA:DP
set_property -dict { PACKAGE_PIN T10     IOSTANDARD LVCMOS33 } [get_ports { sseg[7] }];
set_property -dict { PACKAGE_PIN R10     IOSTANDARD LVCMOS33 } [get_ports { sseg[6] }];
set_property -dict { PACKAGE_PIN K16     IOSTANDARD LVCMOS33 } [get_ports { sseg[5] }];
set_property -dict { PACKAGE_PIN K13     IOSTANDARD LVCMOS33 } [get_ports { sseg[4] }];
set_property -dict { PACKAGE_PIN P15     IOSTANDARD LVCMOS33 } [get_ports { sseg[3] }];
set_property -dict { PACKAGE_PIN T11     IOSTANDARD LVCMOS33 } [get_ports { sseg[2] }];
set_property -dict { PACKAGE_PIN L18     IOSTANDARD LVCMOS33 } [get_ports { sseg[1] }];
set_property -dict { PACKAGE_PIN H15     IOSTANDARD LVCMOS33 } [get_ports { sseg[0] }];

## 7 segment display anodes | an[7:0] -> AN7:AN0
set_property -dict { PACKAGE_PIN J17     IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
set_property -dict { PACKAGE_PIN J18     IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
set_property -dict { PACKAGE_PIN T9      IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
set_property -dict { PACKAGE_PIN J14     IOSTANDARD LVCMOS33 } [get_ports { an[3] }];
set_property -dict { PACKAGE_PIN P14     IOSTANDARD LVCMOS33 } [get_ports { an[4] }];
set_property -dict { PACKAGE_PIN T14     IOSTANDARD LVCMOS33 } [get_ports { an[5] }];
set_property -dict { PACKAGE_PIN K2      IOSTANDARD LVCMOS33 } [get_ports { an[6] }];
set_property -dict { PACKAGE_PIN U13     IOSTANDARD LVCMOS33 } [get_ports { an[7] }];
```

Appendix B

Since the state text is cutoff, the states in order of appearance are reset_state, test_state, increment, reload_term, generate_next_term, shift, mult_add, done_state. Each view is 150 ns.



Entity: three_k_plus_one Architecture: structural Date: Fri Jun 26 06:41:11 PM EDT 2026 Row: 1 Page: 1

